

MANUAL PRÁCTICO · SPEC-DRIVEN DEVELOPMENT

OpenSpec desde cero

Guía paso a paso para adoptar OpenSpec con Claude Code en proyectos
.NET Core + Angular, integrado con Jira y Azure DevOps.

explore › propose › revisión › apply › verify › archive

Índice

EMPEZAR

00	Cómo usar este manual	2
	Ruta recomendada y a quién va dirigido	
01	Entender OpenSpec	3
	Qué es, qué problema resuelve y cuándo usarlo	

MONTARLO

02	Preparar el entorno	5
	Requisitos, instalación y telemetría	
03	Inicializar el repositorio	6
	openspec init, estructura y artefactos	
04	Dar contexto a la IA	8
	CLAUDE.md frente a openspec/config.yaml	

TRABAJAR

05	Escribir specs	10
	Requisitos, escenarios y deltas	
06	Tu primer cambio, paso a paso	13
	Del ticket al merge con un ejemplo completo	
07	Recetas por tipo de cambio	16
	Feature, bugfix, refactor, paralelo, cambios grandes	

ESCALAR

08	Configurar Claude Code para el equipo	17
	MCP, permisos, hooks, modelos y subagente	
09	Integrar con Jira y Azure DevOps	19
	Mapeo, convenciones, MCP, estados y pipeline	
10	Adoptarlo en el equipo	22
	Fases, Definition of Done, métricas y riesgos	

ANEXOS

A	Chuleta de comandos	24
B	Plantillas listas para copiar	26
C	Checklists	28
D	Preguntas frecuentes	29
E	Recursos y glosario	30

Cómo usar este manual

Una ruta práctica para pasar de no haber usado nunca OpenSpec a trabajar con él a diario en proyectos .NET Core + Angular, con Claude Code y un backlog en Jira o Azure DevOps.

A quién va dirigido

A desarrolladores y tech leads que ya trabajan con asistentes de IA y quieren que el agente implemente a partir de especificaciones revisadas, no de prompts sueltos. Se asume nivel senior en .NET y Git; no se asume ningún conocimiento previo de OpenSpec.

Ruta recomendada

El manual está ordenado para leerse de principio a fin la primera vez. Después, los anexos sirven como consulta rápida.

Momento	Qué hacer	Capítulos
Día 1	Entender el modelo mental: specs, changes y deltas	01
Día 2	Instalar, ejecutar <code>openspec init</code> en un repo de pruebas y hacer <code>/opsx:onboard</code>	02, 03
Día 3	Aprender a escribir specs y recorrer las recetas	05, 07
Día 4	Redactar <code>CLAUDE.md</code> y <code>config.yaml</code> reales para un proyecto del equipo	04
Día 5	Primer cambio real y pequeño (bugfix o endpoint)	06
Sprints siguientes	Endurecer la configuración, conectar el backlog y extenderlo al equipo	08, 09, 10

VERSIÓN DE REFERENCIA

Todo el manual está escrito sobre **OpenSpec v1.13.0** (revisión del 28/09/2026). Las versiones anteriores a la 1.x usaban comandos y ficheros distintos (`/openspec:proposal` , `project.md`); los actuales son `/opsx:*` y `openspec/config.yaml` . Ante cualquier discrepancia, manda la carpeta `docs/` del repositorio oficial.

Convenciones del documento

- `openspec ...` son comandos de **terminal** (CLI).
- `/opsx:...` son comandos de **chat** dentro de Claude Code.
- El ejemplo que recorre el manual es una historia real de tipo: «*rechazar solicitudes duplicadas de un alumno para la misma asignatura*», con ticket `PROJ-1234` (Jira) o `AB#1234` (Azure Boards).
- Los cuadros **Objetivo** resumen lo que debes tener al terminar cada capítulo.

Entender OpenSpec

Antes de instalar nada, el modelo mental: qué es una spec, qué es un change y por qué se trabaja con deltas.

Qué es

OpenSpec es un framework ligero y de código abierto (licencia MIT, de Fission-AI) para hacer **spec-driven development (SDD)** con asistentes de IA. Su objetivo es que personas y agentes se pongan de acuerdo sobre **qué** hay que construir antes de escribir código, y que ese acuerdo quede versionado en el repositorio.

El problema que resuelve

Sin especificaciones, el trabajo con agentes se basa en prompts vagos: el agente rellena huecos con suposiciones, el conocimiento se pierde en el historial del chat y nadie puede revisar la intención antes de ver el código. OpenSpec introduce un paso intermedio y revisable: el agente redacta una **propuesta de cambio** (qué, por qué, cómo y lista de tareas) que el equipo aprueba **antes** de implementar.

Los seis principios

Principio	Qué significa en la práctica
Specs como fuente de verdad	<code>openspec/specs/</code> describe cómo se comporta el sistema hoy, organizado por dominios.
Cambios como deltas	Cada cambio vive en su carpeta en <code>openspec/changes/</code> y solo describe lo que añade, modifica o elimina.
Brownfield-first	No hay que documentar todo el sistema antes de empezar: solo se especifica lo que se toca.
Ligero e iterativo	Sin fases rígidas; los artefactos se editan en cualquier momento.
Agnóstico de herramienta	Más de 30 asistentes: Claude Code, Copilot, Cursor, Codex, Gemini CLI, Windsurf, Kiro...
Todo en Git	Specs y cambios se versionan con el código y se revisan en las PR.

El modelo mental en una imagen

openspec/specs/ — cómo es hoy

Fuente de verdad, por dominio funcional (`solicitudes`, `autenticacion`...). **Nunca se edita a mano**: solo cambia al archivar un change.

openspec/changes/<id>/ — lo que va a cambiar

Una carpeta por historia: `proposal.md`, delta specs (ADDED / MODIFIED / REMOVED), `design.md` y `tasks.md`.

El ciclo básico



Al archivar, las delta specs se fusionan en `openspec/specs/` y el change se mueve a `openspec/changes/archive/`.

Por qué OpenSpec y no otra herramienta

Herramienta	Enfoque	Puntos fuertes	Limitaciones
OpenSpec	Ligero, deltas, brownfield-first	Agnóstico de herramienta y modelo; poca ceremonia	Menos guiado en proyectos desde cero
GitHub Spec Kit	Fases rígidas (constitution, specify, plan, tasks)	Muy exhaustivo	Pesado; dependencias Python; orientado a greenfield
Kiro (AWS)	IDE con specs integradas	Experiencia integrada	Atado a su IDE y modelos
Sin specs	Prompts directos	Rápido para prototipos	Impredecible; sin trazabilidad

RECOMENDACIÓN

OpenSpec: trabajamos sobre aplicaciones existentes (.NET Core + Angular) y queremos integrarlo en Scrum sin añadir burocracia.

Qué NO es

- **No es un generador de código:** el código lo genera el agente (Claude Code).
- **No sustituye al backlog:** el backlog guarda el *qué* de negocio y el estado; OpenSpec el *cómo* técnico y el comportamiento.
- **No sustituye a los tests:** cada escenario de una spec debe acabar en un test automatizado.

OBJETIVO DEL CAPÍTULO

Saber explicar la diferencia entre `specs/` y `changes/` y el orden del ciclo `explore → propose → apply → verify → archive`.

Preparar el entorno

Diez minutos: Node, la CLI de OpenSpec y Claude Code.

Requisitos

- ☐ Node.js **20.19.0 o superior** (recomendado Node 22 LTS).
- ☐ **Claude Code**, en terminal o como extensión de VS Code (útil también para Angular).
- ☐ Git.

Instalación

1 Instala la CLI

```
npm install -g @fission-ai/openspec@latest  
openspec --version
```

También con pnpm, bun, yarn o nix. En CI, fija la versión exacta (`@1.13.0`), nunca `@latest` .

2 Desactiva la telemetría

Solo recoge nombre de comando y versión, pero en entorno corporativo conviene desactivarla:

```
openspec config set telemetry.enabled false  
# alternativa por variable de entorno: OPENSPEC_TELEMETRY=0 o DO_NOT_TRACK=1
```

En CI se desactiva automáticamente. El contenido de specs y código lo procesa el modelo de la herramienta de IA, no OpenSpec.

3 Comprueba Claude Code

Abre Claude Code en cualquier repo y confirma que funciona. Para el piloto, lo más sencillo es un plan **Claude Team / Enterprise**; si se prefiere facturar por Azure y mantener el tratamiento en el tenant, Claude vía **Microsoft Foundry**.

ANTES DE USARLO CON CÓDIGO REAL

Valida con el DPO el tratamiento de código y tickets que puedan contener datos personales (RGPD). Es un paso del plan de adopción, no un trámite opcional.

OBJETIVO DEL CAPÍTULO

`openspec --version` devuelve 1.13.x, la telemetría está desactivada y Claude Code arranca.

Inicializar el repositorio

Un único comando genera la estructura; lo importante es entender qué es cada pieza.

Ejecutar init

```
cd MiSolucion
openspec init
```

Durante `init` elige la herramienta de IA: **Claude Code**. OpenSpec genera la carpeta `openspec/` y los comandos/skills en `.claude/`.

NO EDITES LOS COMANDOS GENERADOS
Se sobrescriben con `openspec update`, que hay que ejecutar tras subir de versión. Las personalizaciones van en comandos propios (capítulo 08).

Estructura resultante

```
MiSolucion/
├── .claude/                # comandos/skills de OpenSpec para Claude Code
├── openspec/
│   ├── config.yaml        # contexto y reglas para la IA
│   ├── specs/             # FUENTE DE VERDAD, por dominio
│   │   ├── solicitudes/spec.md
│   │   └── autenticacion/spec.md
│   └── changes/           # cambios en curso
│       ├── proj-1234-rechazo-duplicados/
│       │   ├── proposal.md
│       │   ├── specs/solicitudes/spec.md # delta spec
│       │   ├── design.md
│       │   └── tasks.md
│       └── archive/       # cambios completados
└── src/ ...
```

Los cuatro artefactos de un change

Artefacto	Contenido	Quién lo revisa
proposal.md	Intención, alcance, enfoque y enlace al ticket	PO / Tech Lead
specs/ (delta)	Requisitos ADDED / MODIFIED / REMOVED con escenarios	Equipo — clave en la revisión
design.md	Decisiones técnicas y arquitectónicas	Tech Lead / arquitecto
tasks.md	Checklist de implementación con progreso	Desarrollador

Primer ciclo guiado

Antes de tocar un proyecto real, ejecuta en el chat de Claude Code:

`/opsx:onboard`

Es un tutorial que recorre un cambio completo sobre tu propio código. Es la forma más rápida de interiorizar el flujo.

Comandos de terminal que usarás desde el primer día

Comando	Para qué
<code>openspec list</code>	Ver los changes activos
<code>openspec show <cambio></code>	Detalle de un change
<code>openspec validate <cambio> --strict</code>	Validar estructura y formato
<code>openspec view</code>	Dashboard interactivo

Referencia completa en el Anexo A.

OBJETIVO DEL CAPÍTULO

Repo de pruebas inicializado, estructura entendida y un `/opsx:onboard` completado.

Dar contexto a la IA

La calidad de las propuestas depende casi por completo del contexto. Dos ficheros, dos responsabilidades, sin duplicar.

Reparto de responsabilidades

Fichero	Cuándo se usa	Qué poner
CLAUDE.md	Siempre, en toda la sesión	Stack, arquitectura, convenciones de código, comandos de build/test, reglas del repo
openspec/config.yaml	Al generar artefactos OpenSpec	Glosario de dominio y reglas para proposal, specs, design y tasks

CLAUDE.md: la regla que no puede faltar

Además del stack y las convenciones (plantilla completa en el Anexo B), añade una sección de SDD:

CLAUDE.md (extracto)

```
## Spec-driven development (OpenSpec)
- Todo cambio de comportamiento pasa por un change de OpenSpec (/opsx:propose).
- NUNCA editas openspec/specs/ directamente: solo cambia mediante /opsx:archive.
- Cada Scenario de una spec debe tener un test automatizado.
- Nombre del change = nombre de la rama = <clave-ticket>-<slug>.
```

openspec/config.yaml

`context` se inyecta en **todos** los artefactos; `rules` solo en el artefacto correspondiente.

openspec/config.yaml

schema: spec-driven

context: |

Dominio: gestión académica (expedientes, solicitudes, titulaciones).

Glosario:

- expediente: registro académico de un alumno
- solicitud: trámite iniciado por el alumno

Las APIs públicas mantienen compatibilidad hacia atrás.

rules:

proposal:

- Primera línea: enlace al ticket de origen (Jira PROJ-XXXX o Azure Boards AB#XXXX)
- No reescribir el porqué de negocio; resumirlo y enlazar el ticket
- Incluir impacto en base de datos y plan de rollback
- Identificar contratos de API o eventos afectados (breaking o no)

specs:

- Formato Given/When/Then; un escenario por regla de negocio y por caso de error
- Cada criterio de aceptación del ticket tiene al menos un Scenario
- Si un criterio es ambiguo, registrarlo como pregunta abierta en proposal.md, no inventarlo

design:

- Referenciar patrones existentes antes de proponer nuevos

tasks:

- Cada tarea incluye su test asociado
- Migraciones de base de datos en tarea separada

CRITERIO PRÁCTICO

Si una regla afecta a cómo se escribe *código*, va en `CLAUDE.md`. Si afecta a cómo se escribe una *propuesta* o una *spec*, va en `config.yaml`. Duplicar provoca contradicciones cuando uno de los dos evoluciona.

OBJETIVO DEL CAPÍTULO

`CLAUDE.md` y `config.yaml` redactados con el stack y el dominio **reales** del proyecto piloto, versionados en el repo.

Escribir specs

Las specs son lo que el equipo revisa de verdad. Describen comportamiento observable, no implementación.

Reglas básicas

- Organización **por dominio funcional** (`specs/solicitudes/` , `specs/titulos/`), no por capas técnicas ni por proyecto de la solución.
- Cada **Requirement** usa lenguaje normativo: **SHALL** / **MUST**.
- Cada requisito tiene uno o más **Scenario** en formato **GIVEN** / **WHEN** / **THEN**.
- Nada de nombres de clases o tablas salvo que formen parte del contrato.

Spec principal (fuente de verdad)

```
openspec/specs/solicitudes/spec.md
```

```
# Solicitudes
```

```
## Requirements
```

```
### Requirement: Alta de solicitud
```

```
The system SHALL allow an authenticated alumno to create a solicitud for an  
asignatura in which they are enrolled.
```

```
#### Scenario: Alta correcta
```

- GIVEN an authenticated alumno enrolled in asignatura X
- WHEN the alumno submits a solicitud for X
- THEN the API returns 201 Created with the solicitud id
- AND the solicitud is stored with estado "Pendiente"

```
#### Scenario: Alumno no matriculado
```

- GIVEN an authenticated alumno not enrolled in asignatura X
- WHEN the alumno submits a solicitud for X
- THEN the API returns 422 with code ALUMNO_NO_MATRICULADO

Delta specs: los tres verbos

ADDED — nuevo comportamiento

```
openspec/changes/proj-1234-rechazo-duplicados/specs/solicitudes/spec.md
```

```
## ADDED Requirements
```

```
### Requirement: Rechazo de solicitudes duplicadas
```

```
The system SHALL reject a new solicitud when an active solicitud exists for the same alumno and asignatura.
```

```
#### Scenario: Solicitud duplicada
```

- GIVEN an active solicitud for alumno A in asignatura X
- WHEN alumno A submits a new solicitud for X
- THEN the API returns 409 Conflict with code SOLICITUD_DUPLICADA

```
#### Scenario: Solicitud anterior cerrada
```

- GIVEN a closed solicitud for alumno A in asignatura X
- WHEN alumno A submits a new solicitud for X
- THEN the API returns 201 Created

MODIFIED — cambia un comportamiento existente (típico de un bug)

Se reescribe el requisito **completo**, no solo la diferencia:

```
## MODIFIED Requirements
```

```
### Requirement: Alta de solicitud
```

```
The system SHALL allow an authenticated alumno to create a solicitud for an asignatura in which they are enrolled in the current academic year.
```

```
#### Scenario: Matrícula de un curso anterior
```

- GIVEN an alumno enrolled in asignatura X only in a previous academic year
- WHEN the alumno submits a solicitud for X
- THEN the API returns 422 with code ALUMNO_NO_MATRICULADO

REMOVED — comportamiento que desaparece

```
## REMOVED Requirements
```

```
### Requirement: Envío de solicitud por email
```

```
**Reason**: sustituido por el formulario web; el buzón se da de baja.
```

```
**Migration**: las solicitudes pendientes por email se migran manualmente antes del despliegue.
```

Ocho buenas prácticas

1. **Un escenario por regla de negocio y por caso de error.** Los errores también son comportamiento.
2. **Códigos de error estables** (`SOLICITUD_DUPLICADA`) en lugar de mensajes literales: Angular y los tests dependen de ellos.
3. **Criterios de aceptación → escenarios.** Si un criterio es ambiguo, se pregunta; no se inventa.
4. **Escenarios → tests.** Cada Scenario, un test (xUnit en backend, Jest/Karma en Angular).
5. **Bugs como MODIFIED.** Describe el comportamiento correcto, no «arreglar X». Así el bug queda documentado y no vuelve.
6. **Refactors sin delta.** Si el comportamiento no cambia, no hay delta specs; se archiva con `--skip-specs`.

7. **Idioma.** Palabras normativas (SHALL/MUST, GIVEN/WHEN/THEN) en inglés para que validador y agente las reconozcan; términos de dominio en español. Criterio uniforme en todo el repo.
8. **No backfilling masivo.** Las specs crecen cambio a cambio.

OBJETIVO DEL CAPÍTULO

Saber escribir y revisar una delta spec con la checklist del Anexo C, y que `openspec validate <cambio> --strict` pase.

Tu primer cambio, paso a paso

Recorrido completo de la historia «Rechazar solicitudes duplicadas», desde el ticket hasta el merge a `main`.

Vista general del flujo



1 Parte de un ticket bien escrito

Idealmente con los criterios ya en Dado/Cuando/Entonces (plantilla en el Anexo B) y **sin datos personales reales**:

```

Como alumno quiero que no se me permita duplicar una solicitud
para no generar trámites repetidos.

Criterios de aceptación
1. Dado una solicitud activa para la asignatura X, cuando creo otra para X,
   entonces recibo un error de solicitud duplicada.
2. Dado una solicitud cerrada para X, cuando creo otra para X,
   entonces se crea correctamente.
  
```

2 Crea la rama con la convención de nombres

```
git switch -c proj-1234-rechazo-duplicados # Azure Boards: ab1234-rechazo-duplicados
```

Nombre del change = nombre de la rama. Es la integración más importante con el backlog.

3 Explora si hay incertidumbre

```
/opsx:explore ¿Dónde se valida hoy el alta de solicitudes y qué estados existen?
```

No escribe nada; investiga código y problema. En Claude Code encaja con el *plan mode* (Shift+Tab). Para cambios triviales puedes saltarlo.

4 Genera la propuesta

```
/opsx:propose proj-1234-rechazo-duplicados
```

Crea `proposal.md`, `delta specs`, `design.md` y `tasks.md`, arrastrando el contexto de la exploración. Con el backlog conectado (capítulo 09), usa `/ticket-propose PROJ-1234`, que lee el ticket y convierte cada criterio en escenario.

5 Revisa antes de implementar — el punto de control clave

Tech Lead o equipo revisan: ¿resuelve el problema correcto?, ¿escenarios completos, incluidos errores y límites?, ¿diseño coherente con la arquitectura? Si hay cambios, se itera **sobre el mismo change**. Valida el formato:

```
openspec validate proj-1234-rechazo-duplicados --strict
```

6 Implementa

```
/model sonnet  
/opsx:apply proj-1234-rechazo-duplicados
```

El agente implementa las tareas de `tasks.md` (código + tests) y las va marcando. Con un buen `tasks.md`, `sonnet` es suficiente, más rápido y barato.

7 Verifica

```
/opsx:verify  
Usa el subagente spec-verifier sobre el cambio proj-1234-rechazo-duplicados
```

Comprueba que cada Requirement/Scenario tiene código y test, y detecta tareas marcadas como hechas que no lo están. `/opsx:verify` requiere el perfil ampliado (`openspec config profile`).

8 Archiva dentro de la misma PR

```
/opsx:archive
```

Fusiona las delta specs en `openspec/specs/solicitudes/spec.md` y mueve el change a `changes/archive/`.

EL ARCHIVE VA ANTES DEL MERGE

Código y specs actualizadas deben entrar juntos en `main`. Si se archiva después, hay una ventana en la que la fuente de verdad miente.

9 Abre la PR y haz merge

Commits y PR con `PROJ-1234` o `AB#1234`. La pipeline ejecuta `openspec validate --all --strict + build + tests`. El revisor comprueba código, delta specs y archive. Al hacer merge, la automatización nativa del backlog pasa el ticket a *Done*.

Qué revisar en cada punto de control

Momento	Revisor	Foco
Tras <code>propose</code>	Tech Lead / equipo	¿Problema correcto? ¿Escenarios completos? ¿Diseño coherente con la arquitectura?
Tras <code>verify</code>	Desarrollador	¿Cada escenario tiene código y test? ¿ <code>tasks.md</code> refleja la realidad?
PR	Revisor de código	Código + delta specs + archive; pipeline verde

OBJETIVO DEL CAPÍTULO

Un change real, pequeño, archivado y mergeado en el proyecto piloto.

Recetas por tipo de cambio

No todos los cambios necesitan el ciclo completo. Elige la receta según el tipo de trabajo.

Situación	Secuencia	Clave
1. Funcionalidad pequeña	<code>/opsx:propose</code> → revisar → <code>/opsx:apply</code> → <code>/opsx:archive</code>	Camino rápido; sin explore
2. Bugfix	Igual que la 1	Proponer el comportamiento correcto; delta con MODIFIED
3. Con incertidumbre	<code>/opsx:explore</code> (plan mode) → <code>/opsx:propose</code>	La propuesta hereda el contexto de la exploración
4. Cambios en paralelo	<code>/opsx:apply</code> <code>proj-1250-exportacion-pdf</code>	Cada change en su carpeta; combinar con git worktrees y una sesión de Claude Code por worktree
5. Refactor	Proposal + tasks, sin delta → <code>openspec archive <cambio> --skip-specs</code>	Si no cambia comportamiento, no hay spec que tocar
6. Cambio grande	<code>/opsx:new</code> → <code>/opsx:continue</code> (artefacto a artefacto) → <code>/opsx:ff</code>	Revisar entre artefactos; perfil ampliado
7. Aprender el código	<code>/opsx:onboard</code>	Tutorial guiado sobre el propio repo

Épicas y cambios de alto impacto: PR de specs separada

Abre primero una **PR solo con el change** (proposal + specs + design + tasks), revísala y apruébala, y después implementa en la misma rama o en otra. Para cambios pequeños basta con revisar las specs dentro de la PR de código.

Urgencias sin tiempo para specs

Etiqueta `no-spec` **visible** en la PR (hotfix, configuración). Define la excepción desde el primer día y revisa su uso en las retrospectivas: si se abusa, el proceso se erosiona.

Trabajo en paralelo con worktrees

```
git worktree add ../MiSolucion-1250 -b proj-1250-exportacion-pdf
cd ../MiSolucion-1250 && claude
# en esa sesión:
/opsx:apply proj-1250-exportacion-pdf
```

Configurar Claude Code para el equipo

Todo versionado en el repo para que el equipo trabaje igual. Las preferencias personales, en `.claude/settings.local.json` (no versionado).

Estructura versionada

```
.claude/
├── commands/opsx/...           # generados por openspec init (no editar)
├── commands/ticket-propose.md  # comando propio: ticket → propuesta
├── agents/spec-verifier.md     # subagente de verificación independiente
├── settings.json               # permisos, hooks y variables de entorno
├── .mcp.json                  # servidores MCP (Jira / Azure DevOps), sin secretos
├── CLAUDE.md                  # stack y convenciones de código
└── openspec/config.yaml       # contexto de dominio y reglas de specs
```

Permisos, variables y hook de validación

```
.claude/settings.json
{
  "env": { "OPENSPEC_TELEMETRY": "0" },
  "permissions": {
    "allow": [
      "Bash(openspec list:*)", "Bash(openspec show:*)", "Bash(openspec validate:*)",
      "Bash(dotnet build:*)", "Bash(dotnet test:*)", "Bash(npm run test:*)"
    ],
    "ask": ["Bash(openspec archive:*)", "Bash(git push:*)"],
    "deny": ["Read(./**/appsettings.Production.json)", "Read(./env*)"]
  },
  "hooks": {
    "Stop": [{
      "hooks": [{
        "type": "command",
        "command": "openspec validate --all --strict --no-interactive 1>&2 || exit 2"
      }]
    }]
  }
}
```

- **Hook Stop** : al terminar cada turno se validan las specs; si fallan, el código de salida 2 impide finalizar y devuelve los errores a Claude para que los corrija. Validación local sin esperar a la pipeline.
- **ask para archive y push**: operaciones con efecto en la fuente de verdad o fuera del puesto requieren confirmación.
- **deny** sobre ficheros con secretos o configuración de producción.

Modelos por fase

Fase	Modelo	Cómo fijarlo
Explore / Propose	opus	model: opus en el frontmatter de /ticket-propose ; plan mode para explorar

Apply	<code>sonnet</code>	<code>/model sonnet</code> — suficiente si <code>tasks.md</code> es bueno
Verify	<code>opus</code> en subagente	Contexto limpio: no se autojustifica
Archive	cualquiera	Operación casi mecánica

Usa alias (`opus` , `sonnet`) en lugar de versiones concretas para que la configuración no caduque.

Subagente verificador

`.claude/agents/spec-verifier.md` (plantilla en el Anexo B). Sin modificar nada, revisa que cada Requirement/Scenariio tenga código y test, ejecuta los tests y detecta tareas marcadas que no están hechas. Complementa a `/opsx:verify` con una segunda opinión en contexto limpio.

Comando propio: del ticket a la propuesta

`/ticket-propose PROJ-1234` (o `AB#1234`) lee el ticket vía MCP en solo lectura, se detiene si hay criterios ambiguos, ejecuta `/opsx:propose` con el nombre correcto y presenta un resumen para revisión. Plantilla en el Anexo B.

Más adelante: revisión automática en CI

Una vez asentado el flujo, ejecutar Claude Code en modo no interactivo (`claude -p`) en Azure Pipelines con el prompt del `spec-verifier` y publicar el resultado como comentario en la PR. **No en el primer sprint:** mide antes el valor de la revisión humana y el coste en tokens por PR.

Integrar con Jira y Azure DevOps

Principio rector: no duplicar. El backlog guarda el *qué* y el estado; el repositorio, el *cómo* y el comportamiento.

Backlog (Jira / Azure Boards)

Qué de negocio, criterios de aceptación y **estado**.
Donde trabaja el Product Owner.

Repositorio (OpenSpec)

Cómo técnico y **fuentes de verdad del comportamiento**. Donde trabaja el equipo de desarrollo.

Intentar sincronizar todo en ambos sentidos lleva a divergencias en pocas semanas.

Mapeo de conceptos

Backlog	OpenSpec
Epic / Feature	Dominio en <code>openspec/specs/<dominio>/</code> + varios changes
Story / PBI	Un change (<code>openspec/changes/<id>/</code>)
Criterios de aceptación	Scenarios Given/When/Then en la delta spec
Bug	Change con requisito MODIFIED
Deuda técnica / refactor	Change sin delta specs (<code>archive --skip-specs</code>)
Sub-tasks / Tasks	<code>tasks.md</code> — una sola fuente, no duplicar en el backlog
Estado (To Do → In Progress → Done)	Derivado de eventos de Git mediante automatizaciones nativas

Convención de nombres

- **Change = rama:** `proj-1234-rechazo-solicitudes-duplicadas` (Jira) • `ab1234-rechazo-solicitudes-duplicadas` (Azure Boards).
- **Commits y PR con la clave:** `PROJ-1234` (smart commits) • `AB#1234` (enlace automático al work item).
- **Primera línea de `proposal.md`:** enlace al ticket, forzado con `rules.proposal`.

Conectar el agente al backlog (MCP)

Jira — Atlassian RoVo MCP Server (oficial, remoto)

- Endpoint recomendado: `https://mcp.atlassian.com/v1/mcp/authv2` (OAuth 2.1; cada desarrollador se autentica con su cuenta desde `/mcp`).
- Alternativa con API token: `https://mcp.atlassian.com/v1/mcp`. El endpoint `/v1/sse` está desaconsejado.

Azure DevOps — servidor local de Microsoft

- Paquete `@azure-devops/mcp`, con carga selectiva de dominios y autenticación con `az login`.

- El servidor **remoto** (`https://mcp.dev.azure.com/{organizacion}`) **no admite todavía Claude Code**: Entra no soporta aún el registro dinámico de clientes OAuth.

`.mcp.json` (usa solo el servidor que corresponda al proyecto)

```
{
  "mcpServers": {
    "atlassian": { "type": "http", "url": "https://mcp.atlassian.com/v1/mcp/authv2" },
    "ado": {
      "type": "stdio",
      "command": "npx",
      "args": [ "-y", "@azure-devops/mcp", "${ADO_ORG}",
        "-d", "core", "work", "work-items", "repositories",
        "--authentication", "azcli" ]
    }
  }
}
```

SOLO LECTURA

Consulta los nombres de las tools de escritura con `/mcp` y añádelas a `deny` en `settings.json` (actualizar work items, transicionar issues...). Nunca secretos en `.mcp.json` versionado.

Estados del ticket: automatizaciones nativas, no el agente

Delegar transiciones en el LLM es poco fiable y deja mala trazabilidad. Usa eventos de Git:

- **Jira Automation**: «Branch created → In Progress», «Pull request merged → Done».
- **Azure DevOps**: con `AB#1234` en la PR, activa completar los work items enlazados al completar la PR.

Como mucho, el agente **comenta** en el ticket el enlace a la propuesta, con permisos mínimos.

Validación en la pipeline

`azure-pipelines.openspec.yml`

```
trigger: none
pool: { vmImage: ubuntu-latest }
steps:
- task: NodeTool@0
  inputs: { versionSpec: '22.x' }
- script: npm i -g @fission-ai/openspec@1.13.0
  displayName: Install OpenSpec
- script: openspec validate --all --strict --no-interactive
  displayName: Validate specs
  env: { OPENSPEC_TELEMETRY: '0' }
```

- Con **Azure Repos**, el trigger `pr:` del YAML no aplica: registra la pipeline como **Build Validation** en la branch policy de `main`.
- **Fija la versión** de OpenSpec para builds reproducibles.
- Regla opcional: fallar si la PR modifica `src/` sin tocar `openspec/changes/`, salvo etiqueta `no-spec`.
- Con Jira + Bitbucket/GitHub, el paso es equivalente en Bitbucket Pipelines o GitHub Actions.

Riesgos específicos

- **RGPD:** los tickets pueden contener datos personales (capturas, DNI, emails de alumnos) que el MCP enviaría al modelo. Revisar con el DPO antes de conectar.
- **Divergencia:** si se editan criterios en el ticket tras aprobar la propuesta, hay que actualizar el change o crear uno nuevo.

Adoptarlo en el equipo

Incremental, medible y sin añadir burocracia al flujo Scrum.

Plan por fases

Fase	Duración	Alcance	Criterio de salida
0. Preparación	1 semana	Formación (capítulo 00), elección del piloto, revisión con DPO	Modelo mental; piloto elegido
1. Piloto manual	Sprints 1-2	<code>openspec init</code> , <code>CLAUDE.md</code> , <code>config.yaml</code> , convención de nombres, revisión de specs en PR. Sin MCP	≥ 5 changes archivados; feedback del equipo
2. Integración backlog	Sprint 3	MCP en solo lectura; <code>/ticket-propose</code>	Propuestas desde tickets sin retrabajo mayor
3. Automatización	Sprint 4+	Pipeline con <code>validate</code> , hook <code>Stop</code> , subagente verificador, automatizaciones de estado	Pipeline obligatoria en <code>main</code>
4. Extensión	Posterior	Resto de proyectos; Stores (beta) para specs compartidas front/back; revisión automática en CI	Todos los repos activos

Elegir el proyecto piloto

Aplicación en mantenimiento activo, cobertura de tests razonable, equipo motivado e impacto de negocio medio (no crítico). Empieza por cambios reales y pequeños de la semana: **un bugfix con regla de negocio, un endpoint nuevo con escenarios de error y un refactor**. Cubren las tres variantes de delta (MODIFIED, ADDED y sin delta).

Añadidos a la Definition of Done

- ☐ Change creado y propuesta revisada antes de implementar (o `no-spec` justificado).
- ☐ Cada Scenario cubierto por un test automatizado.
- ☐ Change archivado en la misma PR; `openspec/specs/` actualizado.
- ☐ `openspec validate --strict` en verde.

Métricas del piloto

Métrica	Qué se espera
Retrabajo	Menos iteraciones de código tras la review
Defectos escapados	Menos en historias con spec que sin ella
Tiempo de ciclo	Igual o menor tras la curva de aprendizaje
Cobertura de escenarios	100 % de Scenarios con test
Coste en tokens	Medido por historia

Riesgos y mitigaciones

Riesgo	Mitigación
Revisión de specs «de trámite»	Checklist de revisión (Anexo C); rotar revisores; revisarlo en retros
Backfilling masivo de specs	Política brownfield: solo specs de lo que se toca
Datos personales enviados al modelo	Revisión con DPO; plantilla de ticket sin datos reales; <code>deny</code> de ficheros sensibles
Specs desalineadas con el código	Archive en la PR; validate en pipeline; subagente verificador
Agente modificando el backlog	MCP en solo lectura; estados por automatizaciones nativas
Tutoriales y versiones desactualizadas	Versión fijada; manda <code>docs/</code> del repo; <code>openspec update</code> controlado
Rotura por actualización de OpenSpec	Versión fijada en CI; actualizar en rama dedicada
MCP remoto de Azure DevOps no compatible	Servidor local con <code>az login</code> ; revisar cuando Entra soporte registro dinámico
Cambios sin spec por urgencia	Etiqueta <code>no-spec</code> visible; revisar su uso periódicamente
Coste de tokens	Modelo por fase; medir por historia

Chuleta de comandos

Chat (Claude Code)

Comando	Qué hace	Perfil
<code>/opsx:explore</code>	Analiza código y problema sin escribir nada	core
<code>/opsx:propose <nombre></code>	Crea el change: proposal, delta specs, design, tasks	core
<code>/opsx:apply [<nombre>]</code>	Implementa las tareas de <code>tasks.md</code> y las marca	core
<code>/opsx:sync</code>	Sincroniza el estado del change	core
<code>/opsx:archive</code>	Fusiona deltas en <code>specs/</code> y archiva	core
<code>/opsx:verify</code>	Comprueba que la implementación cumple la spec	ampliado
<code>/opsx:new</code> · <code>/opsx:continue</code> · <code>/opsx:ff</code>	Control incremental: crear, avanzar artefacto a artefacto, fast-forward	ampliado
<code>/opsx:bulk-archive</code>	Archiva varios changes a la vez	ampliado
<code>/opsx:onboard</code>	Tutorial guiado sobre tu propio código	—
<code>/ticket-propose <clave></code>	Comando propio: ticket → propuesta (Anexo B)	propio

Terminal (CLI)

Comando	Uso
<code>openspec init</code>	Inicializa el repositorio
<code>openspec update</code>	Regenera comandos tras actualizar la versión
<code>openspec list</code>	Lista changes activos
<code>openspec show <cambio></code>	Detalle de un change
<code>openspec validate [<cambio>]</code>	Valida estructura y formato (<code>--all</code> , <code>--strict</code> , <code>--no-interactive</code>)
<code>openspec view</code>	Dashboard interactivo
<code>openspec archive <cambio> [--skip-specs]</code>	Archiva; <code>--skip-specs</code> para refactoros
<code>openspec config profile</code>	Cambia el perfil de comandos (core / ampliado)
<code>openspec config set telemetry.enabled false</code>	Desactiva telemetría
<code>openspec schema fork init validate which</code>	Schemas personalizados (avanzado)

Comprueba los flags exactos en `docs/cli.md` para la versión instalada.

Schemas personalizados (cuando domines el flujo por defecto)

```
openspec schema fork spec-driven mi-flujo          # partir del schema por defecto
openspec schema init rapido --artifacts "proposal,tasks"
openspec schema validate mi-flujo
openspec new change mi-cambio --schema mi-flujo
```

Útil para añadir un artefacto ADR o un flujo reducido (solo proposal + tasks) en cambios menores.

Plantillas listas para copiar

Adapta nombres de proyecto, rutas y convenciones antes de usarlas. `config.yaml`, `settings.json`, `.mcp.json` y la pipeline están en los capítulos 04, 08 y 09.

B.1 • CLAUDE.md

CLAUDE.md (raíz del repositorio)

```
# Contexto del proyecto

## Stack
- Backend: .NET 8, ASP.NET Core Web API, Clean Architecture (Api / Application / Domain / Infrastructure)
- Persistencia: EF Core + SQL Server, migraciones code-first
- Mensajería: RabbitMQ (consumidores idempotentes, patrón outbox)
- Frontend: Angular (standalone components, signals)
- Tests: xUnit + FluentAssertions + Testcontainers; Angular con Jest

## Comandos
- Build: `dotnet build`
- Tests backend: `dotnet test`
- Tests frontend: `npm run test` (desde /src/Web)

## Convenciones
- Casos de uso en Application con MediatR; validación con FluentValidation.
- Errores de negocio con códigos estables (p. ej. SOLICITUD_DUPLICADA), mapeados a ProblemDetails.
- Nada de lógica de negocio en controladores.
- Cada migración de EF Core en un commit propio.

## Spec-driven development (OpenSpec)
- Todo cambio de comportamiento pasa por un change de OpenSpec (/opsx:propose).
- NUNCA editas openspec/specs/ directamente: solo cambia mediante /opsx:archive.
- Cada Scenario de una spec debe tener un test automatizado.
- Nombre del change = nombre de la rama = <clave-ticket>-<slug>.
```

B.2 • Comando /ticket-propose

.claude/commands/ticket-propose.md

```
---
description: Crea un cambio OpenSpec a partir de un ticket de Jira o Azure Boards
argument-hint: <PROJ-1234 | AB#1234>
model: opus
---
1. Lee el ticket $ARGUMENTS con el MCP correspondiente (título, descripción, criterios de aceptación, enlaces). Solo lectura: no modifiques el ticket.
2. Si hay criterios ambiguos o contradictorios, lístalos y PARA; pregúntame antes de seguir.
3. Ejecuta /opsx:propose con el nombre `<clave-ticket-en-minúsculas>-<slug-kebab-case>`.
   - proposal.md: enlace al ticket en la primera línea.
   - specs: al menos un Scenario por criterio de aceptación.
4. Muéstrame un resumen de proposal.md y de las delta specs para revisión antes de /opsx:apply.
```

B.3 • Subagente spec-verifier

```
.claude/agents/spec-verifier.md

---
name: spec-verifier
description: Verifica de forma independiente que la implementación de un cambio OpenSpec
  cumple sus specs. Úsalo después de /opsx:apply y antes de abrir la PR.
tools: Read, Grep, Glob, Bash
model: opus
---
No modifiques ningún fichero.
1. Lee openspec/changes/<cambio>/ (proposal, specs, tasks).
2. Para cada Requirement y Scenario, localiza el código que lo implementa y el test que lo cubre.
3. Ejecuta `dotnet test` sobre los proyectos afectados.
4. Devuelve:
  - Tabla: Requirement | Scenario | Código | Test | Estado (cubierto / parcial / sin cubrir)
  - Tareas marcadas [x] en tasks.md que no están realmente implementadas
  - Riesgos detectados (breaking changes, migraciones, seguridad)
```

B.4 • Plantilla de Pull Request

```
## Ticket
PROJ-1234 / AB#1234

## Change OpenSpec
`openspec/changes/archive/<fecha>-proj-1234-...` (o `no-spec` justificado)

## Checklist
- [ ] Propuesta y delta specs revisadas antes de implementar
- [ ] Cada Scenario tiene test automatizado
- [ ] spec-verifier sin elementos "sin cubrir"
- [ ] Change archivado en esta PR (specs/ actualizado)
- [ ] Pipeline verde (openspec validate + build + tests)
- [ ] Migraciones revisadas y con plan de rollback (si aplica)
```

B.5 • Plantilla de ticket (Story / PBI)

```
**Como** <rol> **quiero** <objetivo> **para** <beneficio>

**Criterios de aceptación**
1. Dado <contexto>, cuando <acción>, entonces <resultado>
2. Dado <contexto>, cuando <acción errónea>, entonces <error esperado>

**Fuera de alcance**
- ...

**Notas** (sin datos personales reales)
```

Redactar los criterios ya en Dado/Cuando/Entonces permite al agente convertirlos en escenarios sin interpretar.

Checklists

Arranque del piloto

- ☐ Node $\geq$ 20.19 y OpenSpec instalado en los puestos del equipo.
- ☐ Telemetría desactivada.
- ☐ `openspec init` con Claude Code en el repo piloto.
- ☐ `CLAUDE.md` y `openspec/config.yaml` con el stack y dominio reales.
- ☐ Convención de nombres acordada y documentada.
- ☐ Plantilla de PR actualizada.
- ☐ Definition of Done actualizada.
- ☐ DPO informado.
- ☐ Primer change real elegido para el sprint.

Revisión de una delta spec

- ☐ Cada requisito usa SHALL/MUST y describe comportamiento observable.
- ☐ Todos los criterios de aceptación del ticket están cubiertos por escenarios.
- ☐ Hay escenarios para los casos de error y los límites.
- ☐ Los MODIFIED reescriben el requisito completo, no solo la diferencia.
- ☐ Los REMOVED indican motivo y migración.
- ☐ No hay detalles de implementación que deberían ir en `design.md`.
- ☐ `openspec validate <cambio> --strict` pasa sin errores.

Antes de abrir la PR

- ☐ Rama y change con el mismo nombre (`<clave>-<slug>`).
- ☐ `tasks.md` refleja la realidad; nada marcado que no esté hecho.
- ☐ `spec-verifier` sin elementos «sin cubrir».
- ☐ `/opsx:archive` ejecutado; `openspec/specs/` actualizado en esta PR.
- ☐ Commits y título de la PR con `PROJ-1234 / AB#1234`.
- ☐ Migraciones con plan de rollback, si aplica.

Preguntas frecuentes

Pregunta	Respuesta
¿Tengo que documentar todo el sistema antes de empezar?	No. Solo se escriben specs de lo que se va a cambiar; la documentación crece con cada change archivado.
¿Diferencia entre <code>specs/</code> y <code>changes/</code> ?	<code>specs/</code> describe el sistema hoy; <code>changes/</code> contiene propuestas en curso como deltas. Al archivar, las deltas se fusionan en <code>specs/</code> .
¿Cuándo se archiva?	Antes del merge, dentro de la misma PR.
¿Refactors?	Change con propuesta y tareas, sin deltas; <code>openspec archive <cambio> -skip-specs</code> .
¿Bugs?	Comportamiento correcto como MODIFIED; queda documentado y protegido por un test.
¿Qué modelos uso?	<code>opus</code> para explorar, proponer y verificar; <code>sonnet</code> para implementar.
¿Por qué no el MCP remoto de Azure DevOps?	Autentica con Entra, que aún no soporta el registro dinámico de clientes OAuth que usa Claude Code. El local con <code>az login</code> funciona.
¿Debe el agente mover tickets de estado?	No. Automatizaciones nativas por rama, PR y merge. Como mucho, comenta.
¿Duplico sub-tasks en Jira y <code>tasks.md</code> ?	No. Una sola fuente: <code>tasks.md</code> .
¿Cómo evito specs desactualizadas?	Archive en la PR, validate en pipeline, hook <code>Stop</code> , subagente verificador y regla en <code>CLAUDE.md</code> .
¿Idioma de las specs?	Palabras normativas en inglés; términos de dominio en el idioma del negocio; criterio uniforme.
¿Front y back en repos distintos?	Specs en el repo que define el contrato (backend) o Stores (beta) para specs compartidas.
¿Sustituye a los tests?	No. Las specs dicen qué debe pasar; los tests lo garantizan.
¿Envía datos OpenSpec?	Solo telemetría anónima (comando y versión), desactivable. El contenido lo procesa el modelo de la herramienta de IA.

Recursos y glosario

Documentación oficial

Web: openspec.dev · Repositorio: github.com/Fission-AI/OpenSpec · Manual en [docs/](#) :

Documento	Por qué leerlo
<code>getting-started.md</code>	Instalación, estructura y primer ciclo
<code>overview.md</code>	Modelo mental: specs, changes, deltas
<code>existing-projects.md</code>	Clave: adopción en proyectos existentes
<code>examples.md</code>	Recetas: feature, bugfix, refactor, paralelo
<code>explore.md</code> · <code>workflows.md</code>	Explorar primero y flujos avanzados
<code>commands.md</code> · <code>cli.md</code>	Referencia de comandos de chat y terminal
<code>editing-changes.md</code> · <code>customization.md</code>	Iterar propuestas; <code>config.yaml</code> y schemas
<code>stores-beta/user-guide.md</code>	Specs compartidas entre repositorios

Comunidad

- intent-driven.dev/knowledge/openspec — el recurso externo más completo (vídeos, worktrees, ADRs, TDD/BDD).
- [Linear MCP + OpenSpec](#) — patrón «el qué en el backlog, el cómo en Git».
- [Mapping OpenSpec to Jira](#).
- [Deep dive de arquitectura](#).
- Vídeo recomendado para empezar: [Build with OpenSpec: SDD in Claude Code](#).

Glosario

Término	Definición
SDD	Spec-Driven Development: una especificación estructurada guía la implementación del agente.
Spec	Documento en <code>openspec/specs/<dominio>/</code> con el comportamiento actual (fuente de verdad).
Change	Carpeta en <code>openspec/changes/<id>/</code> con proposal, delta specs, design y tasks.
Delta spec	Fragmento que marca requisitos como ADDED, MODIFIED o REMOVED.
Archive	Fusiona las deltas en <code>specs/</code> y mueve el change a <code>changes/archive/</code> .
MCP	Model Context Protocol: acceso del agente a herramientas externas (Jira, Azure DevOps...).
Brownfield	Proyecto existente, frente a greenfield (desde cero).